

Project: Entropic spring – rediscover Hooke’s law with Monte Carlo

MC sampling, statistical physics, reweighting, scientific visualization

Christian Bierlich, Department of Physics, Lund University,

E-mail: `christian.bierlich@fysik.lu.se`

Summary. You will build a simulation of rubber bands, being pulled by a force, in 1 dimension. You will calculate basic observables (“how stretched is the rubber band?”) as function of force in two ways: 1) by reweighting, and 2) by direct simulation.

This exercise will *not* provide you with a large amount of template code to start with. Rather, it is up to you, to set up reasonable data structures etc. for this project.

The main goal of this exercise is to familiarize you with Monte Carlo methods. You should therefore *not* rely on library implementations of Monte Carlo selection apart from sampling a random number in the interval $R \in [0, 1]$

Learning goals. When you have completed this exercise, you will be able to:

- Translate a simple physical model (the entropic spring) into a working computational simulation using object-oriented programming.
- Apply Monte Carlo sampling techniques to generate and analyze ensembles of microstates, including the use of weighted (reweighted) samples.
- Quantitatively compare Monte Carlo results to analytical predictions, using ratio plots and statistical measures to validate simulations.
- Diagnose and interpret the performance and limitations of Monte Carlo methods, including ensemble overlap and effective sample size.

Physics background

We consider a simple model of a rubber band in one dimension. A chain consisting of $N \gg 1$ links, each of length a . For simplicity, we will assume that all configurations of the chain have the same energy, all that matters for the properties of the chain is how many times it doubles back on itself. We will thus have a number of links pointing in the $+$ -direction, and the rest pointing in the $-$ -direction. If we call the total length of the chain L , and the number of links pointing in the $+$ -direction n , we have:

$$L = a(2n - N). \quad (1)$$

We know from basic thermodynamics, the relation between entropy (S) and force (f) at fixed energy (E) to be:

$$\vec{f} = -T \left(\frac{\partial S}{\partial \vec{x}} \right)_E, \quad (2)$$

at temperature T with $\vec{x}$ being the spatial extension¹. We reduce to one dimension for simplicity:

$$f = -T \left(\frac{\partial S}{\partial L} \right)_E, \quad (3)$$

and our task is now to calculate the microscopic entropy, and see how it changes with L .

¹Note: $\vec{x}$ may be more familiar from the definition of work: $dW = \vec{f}d\vec{x}$: work is equal to force times path.



Figure 1: Ludwig Boltzmann (1844–1906), Austrian physicist and one of the founders of statistical mechanics. He established the microscopic interpretation of entropy, relating it to the number of accessible microstates through eq. (4) This relation is famously engraved on his tombstone in Vienna’s Zentralfriedhof cemetery, as seen in the picture (Credit: Thomas D. Schneider, MIT). Boltzmann’s work laid the foundation for modern thermodynamics, kinetic theory, and statistical physics.

From Boltzmann's tombstone (see fig. 1), we know that the entropy counts the number of microstates corresponds to a given macrostate (Ω):

$$S = k_B \ln(\Omega), \quad (4)$$

with k_B being Boltzmann's constant. A macrostate is specified by the total extension L . Many microstates (different orderings of $\pm$ links) correspond to the same L .

The number of ways to arrange the chain such that n links point in the positive direction, is given by the Binomial coefficient:

$$\Omega(N, n) = \binom{N}{n} = \frac{N!}{n!(N-n)!}. \quad (5)$$

Using Stirling's approximation² we get:

$$\frac{S}{k_B} \approx N \ln(2) - \frac{L^2}{2Na^2}. \quad (6)$$

Math exercise

Fill in the remaining steps of algebra in this derivation. (Hint: First you must apply Stirling's approximation. This gives you two logarithms, which you can Taylor expand in $L/(Na)$ which is always smaller than unity.)

*Math exercises are **not mandatory**, but useful to get more familiar with the derivations. Do the Simulation tasks first.*

Inserting in eq. (3), gives directly:

$$f = \frac{k_B T}{Na^2} L. \quad (7)$$

Comparing to Hooke's law: $f = kL$, we identify an entropic spring constant $k = k_B T / (Na^2)$.

Simulation tasks

Simulation task I: microstates and macrostates

Consider first an ensemble of unbiased microstates. That is, rubber bands which we are not pulling with any force, they are just random configurations.

- Set up reasonable data structures, a `Link` class and a `RubberBand` class both holding relevant parameters. The `RubberBand` class should have a method which calculates the total length L . The constructor of `RubberBand` should take N as a parameter, and randomly sample `Links` pointing in $+$ -or $-$ direction, and store them as a data member.
- Monte Carlo sample a large number of rubber bands. Let $a = 1$ for simplicity, and let $N = 100$.

²Which states that $\ln(n!) \approx n \ln(n) - n$ when n is large.

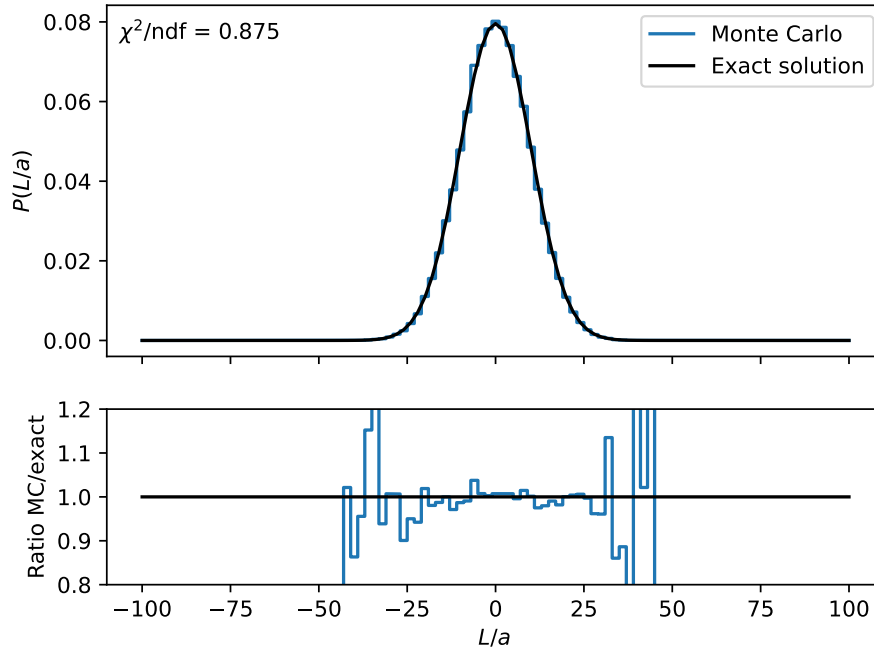


Figure 2: Unbiased rubber band dimensionless length distribution. Each rubber band has $N = 100$ links with $a = 1$. Monte Carlo ($M = 10^6$ samples) histogram is overlaid with the exact solution $P(L) = \Omega(N, n)/2^N$, the ratio plot shows good agreement (although diverging beyond the region of statistical coverage), as does the χ^2/ndf of approximately unity.

- Make a histogram of the $P(L)$ -distribution. Compare to the analytic result: $P(L) = \Omega(N, n)/2^N$ (remember: Ω is the number of microstates with extension L , 2^N is the total number of microstates. The ratio is the probability.)
- Your comparison should (at least) consist of plotting the two overlaid with each other, a plot of the ratio of the two (unity means identical) and some sort of quantitative measure, such as χ^2/ndf . In figure 2 you can see what the figure should look like, with inspiration for how to plot and present it.

Simulation task II: pulling with a force, reweighted

We now introduce an external force f pulling the rubber band in the positive (+) direction. In the previous task you constructed a Monte Carlo sampling of rubber bands unaffected (unbiased) by a force, here we will treat a biased distributed. We will do it using a *reweighting technique*. This means that you do not need to resample the rubber bands, just calculate a new weight with which they enter.

Physical idea: Let E be the energy of a single microstate. We still assume that its internal energy remain constant (E_0). Applying a constant force f along the $+$ -direction, a configuration with extension L does work $-fL$, so its total energy is:

$$E(\text{microstate with } L) = E_0 - fL. \quad (8)$$

At temperature T , such a state has the Boltzmann weight (with $\beta = (k_B T)^{-1}$):

$$w \propto e^{-\beta E} = e^{-\beta E_0} e^{\beta f L}. \quad (9)$$

We already know, that for a given macrostate of length L , there are exactly $\Omega(L)$ corresponding microstates. Adding all their weights together, gives the macrostate weight:

$$W(L) \propto \Omega(L) e^{\beta f L}. \quad (10)$$

Finally, the probability to obtain a given macrostate L , can be found by normalizing over all possible L , here dummy-labelled L' :

$$P(L) = \frac{\Omega(L) e^{\beta f L}}{\sum_{L'} \Omega(L') e^{\beta f L'}}. \quad (11)$$

- Extend your `RubberBand` class with a method that calculates the Boltzmann weight of a given microstate when a force f is applied:

$$w(\text{microstate}) = e^{\beta f L}, \quad \beta = \frac{1}{k_B T}.$$

Here L is the total length calculated by your method from Task I. Hint: Work in units where $k_B = 1$, and start with $T = 1$ such that $\beta = 1$.

- Generate a large ensemble of microstates (still unbiased, random $\pm$ -configurations).
- Construct a *weighted histogram* of the L distribution, where the microstate weight is used. This simulates what happens if the ensemble is pulled with force f .
- Compare your weighted histogram to the analytic result:

$$P_f(L) = \frac{\Omega(N, n) e^{\beta f L}}{Z(f)}, \quad Z(f) = \sum_L \Omega(N, n) e^{\beta f L}.$$

Here $\Omega(N, n)$ is the same combinatorial factor you used in Task I, and $n = (L/a + N)/2$ from inversion of eq. (1).

- Make a plot comparing your Monte Carlo $P_f(L)$ and the analytic $P_f(L)$ for at three small nonzero value of f , e.g. $f = \{0.01, 0.05, 0.1\}$. Again, plot their ratio. Be careful with χ^2 since it is poorly defined for reweighted samples. I recommend skipping it here.
- Try three larger values, e.g. $f = \{0.1, 0.5, 1.0\}$. Do you still see ok performance? Why not? Diagnose your reweighting by calculating the number of effective samples:

$$\mu_{\text{eff}} = \frac{1}{\sum_i \tilde{w}_i^2} \text{ with: } \tilde{w}_i = \frac{w_i}{\sum_j w_j}.$$

Plot μ_{eff}/M as function of the force. When would you expect reweighting to start struggling?

Simulation task III: pulling with a force, direct simulation

As you saw in the previous task, reweighting works only as long as the unbiased and biased ensembles overlap sufficiently. When the force grows too large, only very few unbiased configurations contribute significant weight. To obtain reliable results at larger f , you must instead generate the configurations directly from the biased distribution.

Physical idea: When a constant external force f is applied, each link in the rubber band experiences a bias: it is more likely to point in the $+$ -direction than in the $-$ -direction. From the Boltzmann factor for a single link, the probability for a link to point in the $+$ -direction is:

$$p_+(f) = \frac{w_+}{w_+ + w_-} = \frac{e^{\beta f a}}{e^{\beta f a} + e^{-\beta f a}} = \frac{1}{2}(1 + \tanh(\beta f a)), \quad (12)$$

where $\tanh$ in the last equality is the hyperbolic tangent function. Correspondingly:

$$p_-(f) = 1 - p_+(f). \quad (13)$$

Thus, for a given f you can sample each link independently with these probabilities, instead of assigning random $\pm$ directions with equal weight.

Math exercise

Fill in the algebra needed to write the last equality in eq. (12). (Hint: Look up the definition of $\tanh(x)$ in terms of exponentials).

*Math exercises are **not mandatory**, but useful to get more familiar with the derivations. Do the Simulation tasks first.*

With that in place, we can calculate the expectation value for our upcoming “observable”: $\langle L \rangle(f)$, the (average value) of length given a force. It is simply:

$$\langle L \rangle(f) = a \cdot (\text{number of links in the } + \text{-direction} - \text{number of links in the } - \text{-direction}) \quad (14)$$

$$= aN(p_+ - p_-) = Na(2p_+ - 1) = Na \tanh(\beta f a). \quad (15)$$

In the small-force limit $\beta f a \ll 1$, we can Taylor expand $\tanh(\beta f a) \approx \beta f a$ such that:

$$\langle L \rangle(f) \approx \frac{Na^2}{k_B T} f, \quad (16)$$

matching exactly what was found in eq. (7). The task is now to Monte Carlo simulate a large number of biased states and compare the results to theory.

- Change the constructor of `RubberBand` adding optional arguments `kBT` and `force`. Now your `Link` directions should be sampled from the biased distribution instead of unbiased³.
- Generate a large ensemble (not as large as before, $M = 5000$ should be enough) of rubber bands at several values of f using this biased sampling. For each f , compute the mean extension:

$$\langle L \rangle(f) = \frac{1}{M} \sum_i^M L_i(f),$$

and its standard deviation.

³Note that eq. (12) directly gives a probability to be $+$ as function of the given parameters. Instead of being $+$ with $p = 0.5$, the link should now be $+$ with probability $p_+(f)$.

- Plot $\langle L \rangle$ as a function of $f \in [0, 2.5]$. Compare to the analytic results derived in eq. (14).
- Use your simulation to extract the effective spring constant k_{eff} by fitting $\langle L \rangle$ vs. f in the linear region. Does k_{eff} agree with the theoretical value $k_B T / (Na^2)$? (you could use the error from the fit to make the agreement quantitative, but you don't have to)
- Finally: what happens when f becomes large? Show that the distribution $P_f(L)$ becomes narrow and strongly shifted, and that the linear approximation (Hooke's law) breaks down. You can do this by plotting the standard deviation as a shaded area around the mean, and plot the linear fit along with it.

One of the final points, that the distribution becomes more narrow as the force increases, may not be obvious. However, we can also show this analytically. Recall the calculation of the average in eq. (14). We can calculate the variance of the L distribution as the force increases using similar reasoning. Since $L = a(2n - N)$ and n follows a binomial distribution⁴, we obtain⁵:

$$\sigma^2(L) = \sigma^2(a(2n - N)) = 4a^2\sigma^2(n) = 4a^2Np_+(1 - p_+). \quad (17)$$

Inserting p_+ from eq. (12) and applying a bit of algebra gives:

$$\sigma^2(L) = \frac{a^2N}{\cosh^2(\beta fa)}. \quad (18)$$

As the force grows, $\cosh(\beta fa)$ increases and $\sigma^2(L)$ decreases: the chain becomes increasingly aligned with the force.

Math exercise

Complete the derivation of eq.(18), using the hints given. Remember to look in the footnotes for the variance of a binomial and properties of variance, if you don't know these by heart.

*Math exercises are **not mandatory**, but useful to get more familiar with the derivations. Do the Simulation tasks first.*

For someone used to handling statistical estimators, this may be an easy derivation. Others can note that with Monte Carlo, it was no more difficult to calculate the variance than the mean. This is one of the true powers of Monte Carlo: once your simulation is set up, you can compute any observable from its output.

Simulation task IV: Nearest-neighbor corrections

The first three tasks establish a complete simulation of the entropic spring and its response to an external force. Completing these tasks carefully, with proper validation and discussion, constitutes a full project.

Task IV extends the model by introducing correlations between neighboring links. It builds directly on the techniques from the workshop (reweighting and effective sample size) and is the natural next step if you finish Tasks I–III comfortably.

⁴The variance of a binomial distribution with N trials and probability p of success is $\sigma^2 = Np(1 - p)$.

⁵Variance is invariant under constant shifts and scales quadratically under rescaling.

You are encouraged to attempt Task IV if time allows. It is not required for passing the course, but it is an excellent opportunity to deepen your understanding of statistical mechanics and Monte Carlo methods.

Real polymers are not made up of completely freely flipping links. Neighboring links tend to align with each other, giving the polymer a stiffness. We can model this by correcting the microstate energy of eq. (8) by a term penalizing bends. To do so, we introduce a notation for the links. Let each link be denoted by s_i , where:

$$s_i = \begin{cases} 1 & \text{when pointing in the } + \text{ direction} \\ -1 & \text{when pointing in the } - \text{ direction.} \end{cases} \quad (19)$$

Let neighboring links (i and $i + 1$) interact by adding a term to the energy $-Js_i s_{i+1}$. J is the “stiffness”, which here is ≥ 0 . A large J means that aligned neighboring links are energetically favored, while oppositely oriented links are penalized. The corrected version of the microstate energy now reads:

$$E(\{s_i\}) = E_0 - fL - J \sum_{i=1}^{N-1} s_i s_{i+1}, \text{ with: } L = a \sum_{i=1}^N s_i. \quad (20)$$

The Boltzmann weight of such a state is:

$$w = e^{-\beta E} \propto e^{\beta(fL + J \sum_{i=1}^{N-1} s_i s_{i+1})}. \quad (21)$$

This means that we can reweight our microstates to ones with correlations. In the following we will generate samples at fixed f with direct sampling (like you implemented in the previous task) and reweight only in J . The part of the weight including the force therefore drops out. From an implementation point of view, this is structurally identical to Task II: only the definition of the weight changes.

- Start by revisiting your `Link` class, and ensure that you are storing the links directions as -1 or $+1$ in the negative and positive directions respectively, to mirror the definition of s_i .
- Make a derived class of your `RubberBand` class, call it e.g. `CorrelatedRubberBand`, where you override your previously implemented method for calculating the weight when a force is applied to return:

$$w(\text{microstate}) = e^{\beta J \sum_{i=1}^{N-1} s_i s_{i+1}}.$$

The power of a derived class with an overridden weight method is, that you can now *reuse all your previous code* for analyzing reweighted samples, only replacing `RubberBand` with `CorrelatedRubberBand`.

- Investigate the behaviour of combining generation with fixed f (either positive or negative) and changing J . You can plot the L distribution in a few cases and compare to the theoretical curve at the same fixed f but with no J . Show that the presence of a $J > 0$ seems to enhance and slightly skew the pull regardless of the sign of f . What is the physical mechanism?
- Select a value of f , vary J and plot μ_{eff}/M vs J . How much can we vary J using reweighting? Can you think of ideas to improve our coverage?

Epilogue: Realistic models

The model you have just simulated in the final task is equivalent to the one-dimensional *Ising model*, originally invented to describe magnetism in materials. In that context, each link $s_i = \pm 1$ represents a tiny magnetic moment (a “spin”) that can point up or down, while the parameter J describes how strongly neighboring spins prefer to align. The external force f in our rubber band plays exactly the same role as an external magnetic field.

In polymer physics, this simple model already captures important qualitative behavior. Real molecules such as DNA are not freely jointed: chemical and electrostatic interactions give them a *bending stiffness*. Neighboring molecular segments tend to align over some typical distance before they bend, leading to what is called a **persistence length**. In our one-dimensional model, the persistence length is closely related to how long segments of consecutive links point in the same direction — this is the *correlation length* of the model.

Analytic solutions for correlated systems like this quickly become difficult. The one-dimensional Ising model can still be solved exactly with advanced methods, but already in two dimensions the solution — found by Lars Onsager in 1944 — is a classic milestone in theoretical physics. In three dimensions, no exact solution is known even today.

From the computational side, however, extending the model to higher dimensions is entirely feasible. The underlying Monte Carlo logic remains the same: generate configurations, assign weights, and compute observables as averages. Only the bookkeeping (neighbors, boundaries, and visualization) becomes more involved. Modern simulations of polymers, proteins, or magnetic materials rely on exactly these ideas, just implemented efficiently and in more complex geometries.

Looking forward, this small project has given you a foundation for exploring far richer systems: from polymers and biophysics to phase transitions and materials science. The same statistical reasoning and sampling methods scale far beyond a one-dimensional rubber band — into the models that describe the microscopic structure of our physical world.

What to submit?

Readable, documented code in a merge request to the exercise project. You should clearly describe your results and the methods used. It must include at least:

- **Code:**

- Classes for `Link` and `RubberBand`.
- Cleanly separable code for solving the different simulation tasks.
- Reusable code for plotting comparisons with theory with a ratio plot.
- Code for producing all comparison plots, reweighting plots and direct simulation plots.
- (Task IV only) A derived class `CorrelatedRubberBand` with a correctly implemented nearest-neighbor weight method, and demonstration of its use for reweighting in J .
- Information about how to run the code, documentation of dependencies if needed.

- **Documentation:**

- A clear description of the problem (no need to go through all the theory, but an overview).
- Description of the theoretical curves you compare to.
- Clear descriptions of the use of both direct simulation and reweighted techniques.
- A discussion of when reweighted methods break down.

- (Task IV only) For the correlated case, a short discussion of how introducing a stiffness parameter J affects the L distribution. Include at least one example plot illustrating the change in $P(L)$ as J increases, and one plot of μ_{eff}/M versus J .
- (Task IV only) A brief interpretation of the physical meaning of J , relating it to stiffness or alignment of neighboring links (no detailed derivation needed).

Finished early? Stretch goals

If you complete the core tasks ahead of time, pick one or more of the extensions below. Document what you changed and, if relevant, how it affected your results or the runtime of your code. You are also free to choose a stretch goal on your own, for example exploring the Task IV extension of the model in more detail.

Python is so slow! (optimization, C++)

You may have noticed that simulating a large number of rubber bands can be painfully slow in `Python`. Delegate the simulation task to `C++` and call it through an API (see the 2nd day `Python` tutorial from the beginning of the course). You should deliver:

- `C++` code callable from `Python` with clean instructions for how to build and run through the API.
- Documentation of runtime improvements, for example number of rubber bands generated per clock time in `Python` vs. `C++`.

Animate a rubber band (visualization)

Create a short animation of a single rubber band as the force increases, showing the random links gradually aligning with the force direction. Let “time” in the animation be the force, which is gradually building up. Your animation should show the individual links. You should deliver:

- Runnable code to produce the animation.
- A `.gif` file uploaded to the gitlab.
- Explanation of your process and the animation in the documentation.

Happy Hacking!